



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Никола Огњеновић

**Имплементација погона за
дводимензионалне видео игре
користећи ЕКС архитектуру и
програмски језик Rust**

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Никола Огњеновић	Број индекса:	SV 51 / 2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Имплементација погона за дводимензионалне видео игре користећи ЕКС архитектуру и програмски језик Rust
--

ТЕКСТ ЗАДАТКА:

Циљ овог завршног рада је пројектовање и имплементација модуларног погона за дводимензионалне видео игре у програмском језику <i>Rust</i>, са фокусом на коришћење искључиво безбедног кода у апликативном слоју. Потребно је развити сопствено ЕКС (ентитет-компонента-систем) језгро са подршком за управљање животним циклусом ентитета преко генерација, типизовано складиштење компоненти, систем догађаја, извршавање система и серијализацију стања света. За потребе графичког приказа неопходно је реализовати рендеринг слој заснован на библиотеци <i>wgpu</i>, који омогућава ефикасно беч исцртавање спрајтова и хардверско убрзање путем модерних графичких <i>API</i>-ја. Исправност архитектуре потребно је верификовати јединичним тестовима и практичним примером у виду текстуалне игре засноване на потезима. На крају, кроз наменски развијену евалуациону апликацију извршити стрес тестирање и анализу перформанси погона при раду са великим бројем симулираних ентитета. При изради користити препоручену праксу из области софтверског инжењерства. Детаљно документовати решење.

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - бечелор рад
Аутор, АУ:	Никола Огњеновић
Ментор, МН:	Др Игор Дејановић, редовни професор
Наслов рада, НР:	Имплементација погона за дводимензионалне видео игре користећи ЕКС архитектуру и програмски језик Rust
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	7/41/16/0/1/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	видео игре, Rust, ECS архитектура, погони за видео игре, рендеринг, перформансе
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Овај завршни рад описује имплементацију погона за дводимензионалне видео игре, развијеног у програмском језику Rust. Главни циљ је постизање високих перформанси уз стриктно поштовање безбедносних гаранција које пружа Rust, посебно кроз избегавање unsafe кода. Рад представља имплементацију ЕКС (Entity Component System) архитектуре која омогућава модуларност и ефикасну обраду података. Истражују се различити приступи рендеровању уз коришћење WebGPU стандарда, уз анализу перформанси развијеног погона.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	10.09.2026
Чланови комисије, КО:	Председник: Др Гордана Милосављевић, редовни професор
	Члан: Др Никола Лубурић, ванредни професор
	Члан:
Члан, ментор:	Др Игор Дејановић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Nikola Ognjenović
Mentor, MN :	Igor Dejanović, Phd., full professor
Title, TI :	Implementation of a game engine for two-dimensional video games using ECS architecture and the Rust programming language
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	7/41/16/0/1/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	video games, Rust, ECS architecture, game engines, rendering, performance
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This thesis describes the implementation of a 2D game engine developed in the Rust programming language. The main objective is to achieve high performance while strictly adhering to the safety guarantees provided by Rust, particularly by avoiding <code>unsafe</code> code. The thesis presents the implementation of an Entity Component System (ECS) architecture that enables modularity and efficient data processing. Various rendering approaches using the WebGPU standard are explored, along with an analysis of the performance of the developed game engine.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	10.09.2026
Defended Board, DB :	President: Gordana Milosavljević, Phd., full professor
	Member: Nikola Luburić, Phd., assoc. professor
	Member:
Member, Mentor:	Igor Dejanović, Phd., full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Структура рада	1
2	Стање у области	3
2.1	Погони за видео игре	3
2.2	ЕКС архитектура	3
2.2.1	Зашто се користи ЕКС	4
2.2.2	Поређење ЕКС приступа са алтернативама	4
2.3	Савремени погони за видео игре	4
2.3.1	Unity	5
2.3.2	Unreal Engine	5
2.3.3	Godot	5
2.3.4	Bevy	5
2.4	Разлози избора програмског језика Rust	5
2.4.1	Меморијска безбедност	6
2.4.2	Поређење са C и C++ приступом	6
2.4.3	Улога безбедног и небезбедног Rust-a	6
2.4.4	Могућност развоја погона без небезбедног кода	7
3	Имплементација ЕКС језгра у Rusty погону	9
3.1	Преглед датотека у Rusty погону	9
3.2	lib.rs: модулска декларација и јавни интерфејс	9
3.3	entity.rs: идентификатори, генерације и рециклажа ентитета	10
3.3.1	Зашто не забранити рециклажу идентификатора?	11
3.3.2	Зашто генерације, а не само идентификатори?	11
3.3.3	Зашто не један u64?	11
3.3.4	Зашто не UUID?	11
3.3.5	Инжењерски компромис	11
3.4	component.rs: типизоване компоненте преко type erasure слоја	11
3.4.1	Trait Component	11
3.4.2	Брисање типа складишта компоненти и динамички диспеч	12
3.4.3	HashMapComponentStorage<T: Component> и ComponentManager	13
3.4.4	Избор HashMap модела за складишта компоненти и његове алтернативе	13
3.5	event.rs: систем догађаја са редовима по типу	14
3.6	system.rs: системи и извршни редослед	14
3.7	world.rs: интеграциони слој ЕКС језгра погона	15
3.7.1	Животни циклус ентитета у World-y	15
3.7.2	Серијализација и десеријализација света	15
3.8	Пример примене: текстуална борбена игра	16
4	Опције за рендеринг слој у Rusty погону	19
4.1	Графичке библиотеке ниског нивоа	19
4.1.1	ash	19
4.1.2	vulkano	19
4.2	Апстрактционе библиотеке	20
4.2.1	pixels	20
4.2.2	softbuffer	20

4.3	Наменске библиотеке за дводимензионалне игре	20
4.3.1	<i>ggez</i>	20
4.3.2	<i>macroquad</i>	21
4.3.3	<i>raylib-rs</i>	21
4.4	Аргументација одабира <i>wgpu</i>	22
4.4.1	Мултиплатформска подршка без небезбедног кода у апликативном слоју	22
4.4.2	Директна интеграција са <i>Rust</i> власничким моделом	22
4.4.3	Прилагодљивост рендеринг тока	22
4.4.4	Активно одржавање и широка употреба у екосистему	22
5	Имплементација рендеринг слоја уз <i>wgpu</i>	23
5.1	Архитектура рендеринг слоја	23
5.2	Основе <i>wgpu</i> ресурсног модела	23
5.2.1	Вертексни и индексни бафери	23
5.2.2	Рендеринг пајплајн	23
5.2.3	Везивне групе	23
5.3	Систем ресурса и текстура	24
5.4	Интеграција са <i>EKS</i> компонентама	24
5.5	<i>RenderSystem</i> и петља рендеровања	24
5.6	Употреба <i>wgpu</i> за графичко убрзање	24
6	Евалуациона апликација и тестирање перформанси	27
6.1	Функционалности евалуационе апликације	27
6.2	Техничка имплементација евалуације	27
6.3	Анализа перформанси	27
7	Закључак	29
7.1	Могућности за проширење система	29
7.2	Конкретна примена у виду развоја видео игре	29
	Списак слика	31
	Списак листинга	33
	Списак коришћених скраћеница	35
	Списак коришћених појмова	37
	Биографија	39
	Литература	41

Развој савремених видео игара захтева коришћење или изградњу сложених софтверских система који морају истовремено да задовоље захтеве флексибилности, перформанси и одрживости кода. Како пројекти расту, повећава се број међусобно повезаних подсистема, попут логике игре, управљања ресурсима, физичке симулације и рендеринга. Уколико архитектура није пажљиво осмишљена, временом долази до јаке спреге између различитих делова система, што отежава развој нових функционалности, тестирање и дугорочно одржавање пројекта.

Традиционални приступи засновани на дубоким хијерархијама класа дуго су представљали доминантан модел организације кода у развоју игара. Иако су погодни за мање пројекте, код сложенијих система често доводе до проблема као што су наслеђивање непотребних функционалности, тешкоће при поновној употреби кода и повећана зависност између компоненти. Као алтернатива таквом приступу, последњих година све већу популарност стиче *ЕКС* архитектура заснована на ентитетима, компонентама и системима (енг. *ECS, Entity Component System*), која омогућава јасније раздвајање података од логике и лакше скалирање сложених симулација.

Овај рад се бави развојем погона за дводимензионалне видео игре *Rusty*, заснованог на *ЕКС* архитектури у програмском језику *Rust*. Полазна идеја пројекта била је да се испита у којој мери је могуће изградити функционалан *ЕКС* погон користећи искључиво безбедне механизме које *Rust* нуди, без ослањања на небезбедан (енг. *unsafe*) код. Поред провере техничке изводљивости таквог приступа, циљ је био и да се анализирају његове последице по архитектуру система, једноставност имплементације и перформансе.

Рад обухвата развој кључних делова погона и њихову примену у пратећим програмима. За потребе тестирања *ЕКС* језгра погона развијена је текстуална игра заснована на потезима, која демонстрира рад ентитета, компоненти и система. Поред тога, погон садржи рендеринг систем заснован на библиотеци *wgpu*, која омогућава приказ дводимензионалне сцене коришћењем савремених графичких *АПИ*-ја (апликациони програмски интерфејс, енг. *API, application programming interface*). У оквиру пројекта реализована је и евалуациона апликација којом се испитују границе перформанси система, кроз креирање великог броја ентитета и измену њихове брзине кретања по прозору. Перформансе се процењују на основу вредности *ФПС*-а, (енг. *FPS, frames per second*).

Кроз наредна поглавља биће представљени теоријски концепти на којима је заснован развој погона, донете архитектуралне одлуке, имплементациони детаљи, као и резултати евалуације који омогућавају сагледавање предности и ограничења развијеног решења.

1.1 Структура рада

Рад је подељен у више поглавља, почевши од стања у индустрији, преко теоријских основа, до практичне реализације и евалуације:

- у другом поглављу дат је преглед стања у индустрији што се тиче погона за видео игре, њихових врста и тренутно популарних алата, описана је *ЕКС* архитектура, њене предности у односу на алтернативне приступе и разлози због којих је *Rust* изабран као програмски језик за развој *Rusty* погона;
- у трећем поглављу описана је *ЕКС* архитектура погона за видео игре *Rusty*;
- у четвртом поглављу представљене су опције за имплементацију рендеринг слоја погона *Rusty* и образложен је избор библиотеке *wgpu*;
- у петом поглављу описани су рендеринг систем, детаљи рада са *wgpu* библиотеком и прелаз из *ЕКС* података ка представи на графичком процесору;

- у шестом поглављу приказани су дизајн евалуационе апликације, методологија мерења и интерпретација добијених перформансних показатеља;
- у седмом, закључном поглављу, сумирани су резултати рада, размотрена ограничења система и предложени могући правци даљег развоја *Rusty* погона за видео игре.

2.1 Погони за видео игре

Погон за видео игре (енг. *game engine*) представља софтверски систем који обезбеђује основну инфраструктуру за развој видео игара. Уместо да свака игра засебно имплементира функционалности као што су рендеровање, управљање ресурсима, обрада улаза, физика или звук, ове могућности су обједињене у оквиру погона и доступне су свим пројектима који се на њему развијају.

Савремени погони за видео игре представљају слој апстракције између оперативног система и апликативне логике игре. Они обједињују више подсистема који заједно омогућавају извршавање игре у реалном времену. Типичан погон обухвата:

- систем за рендеровање графике,
- систем за учитавање и управљање ресурсима,
- управљање сценом,
- обраду корисничког улаза,
- систем звука,
- систем за физику и колизије,
- скриптовање или програмски интерфејс,
- алате за развој и дебаговање.

Историјски посматрано, раније видео игре често нису користиле издвојене погоне. Логика игре и инфраструктурни код били су тесно повезани у једној апликацији. Како су пројекти постајали сложенији, појавила се потреба за поновном употребом истих механизма кроз више различитих игара. Из тог разлога постепено се формира концепт погона као засебног слоја који омогућава раздвајање доменске логике игре од инфраструктурних компоненти.

Данас су погони за видео игре један од централних елемената индустрије, јер значајно смањују време развоја и омогућавају тимовима да се фокусирају на дизајн и имплементацију конкретне игре уместо на поновно решавање истих техничких проблема.

2.2 ЕКС архитектура

ЕКС је архитектурални образац који је постао широко прихваћен у модерном развоју видео игара. Основна идеја ЕКС-а јесте раздвајање идентитета објеката, њихових података и логике која те податке обрађује.

ЕКС организује срж погона у три основна слоја:

- ентитети представљају идентификаторе,
- компоненте представљају податке,
- системи представљају логику обраде.

Ентитет обично не садржи никакво понашање нити податке. Он представља само идентификатор над којим се повезују компоненте.

Компоненте су структуре података које описују одређен аспект ентитета. На пример, позиција, брзина, здравље или визуелни приказ могу бити засебне компоненте.

Системи извршавају алгоритме над групама компоненти. Систем за кретање може истовремено обрађивати све ентитете који поседују компоненте позиције и брзине, без потребе да зна било шта о осталим својствима тих ентитета.

Према дефиницији *ЕКС* архитектуре, она је заснована на принципу композиције уместо наслеђивања, и омогућава да се понашање објекта формира комбиновањем компоненти уместо креирањем дубоких хијерархија класа [1].

2.2.1 Зашто се користи *ЕКС*

Главни разлози због којих се *ЕКС* користи у развоју видео игара су:

- већа флексибилност композиције понашања,
- мања повезаност између делова система,
- лакше проширење функционалности,
- погодност за паралелну обраду,
- боље искоришћење процесорског кеша,
- једноставније управљање великим бројем ентитета.

Уместо да сваки објекат садржи комплетну логику и стање, системи обрађују хомогене скупове података. То значајно поједностављује додавање нових функционалности, јер је често за то довољно дефинисати нову компоненту или систем, без измене постојећих структура.

2.2.2 Поређење *ЕКС* приступа са алтернативама

Најчешћа алтернатива *ЕКС* приступу је класичан објектно оријентисан модел.

У класичном моделу објекат истовремено садржи и стање и понашање. Како број различитих типова објеката расте, хијерархије класа постају све дубље и сложеније.

Типичан пример је ситуација у којој постоје класе:

- *GameObject*,
- *Character*,
- *Enemy*,
- *FlyingEnemy*,
- *BossEnemy*.

Временом се појављује велики број комбинација понашања које је тешко изразити искључиво наслеђивањем.

ЕКС приступ избегава овај проблем јер омогућава да се понашање формира композицијом компоненти. Ентитет који има компоненте *Transform*, *Renderable* и *Health* може представљати један тип објекта, док ентитет који има *Transform*, *Renderable*, *Health* и *AI* представља други, без потребе за новим нивоом наслеђивања.

За архитектуралну основу *Rusty* погона изабран је *ЕКС* јер:

- раздваја податке и логику,
- омогућава лако проширење система,
- поједностављује тестирање,
- погодан је за *data-oriented* организацију података,
- природно се уклапа у *Rust* модел власништва и позајмљивања.

2.3 Савремени погони за видео игре

Погони за видео игре се могу категорисати у три основне групе:

- комерцијални општенаменски погони,

- погони и радни оквири отвореног кода (енг. *open source*),
- наменски или интерни погони.

Комерцијални погони су комплетна решења која најчешће укључују визуелне едиторе, системе за изградњу пројекта, интегрисане алате за профилисање и велики број готових пакета.

Отворени погони и радни оквири представљају системе чији је изворни код јавно доступан, што омогућава корисницима да анализирају, модификују и проширују њихову функционалност. За разлику од комерцијалних решења, која нагласак стављају на интегрисано развојно окружење и готове алате, отворени погони често пружају већу флексибилност у погледу архитектурних измена, избора подсистема и прилагођавања специфичним захтевима пројекта.

Интерни погони настају у оквиру студија који развијају игре специфичних захтева. Такви системи су обично прилагођени конкретном жанру или типу пројекта, и често не морају да решавају све проблеме које покривају општенаменски погони.

Најрелевантнији представници данашње индустрије погона за видео игре су *Unity*, *Unreal Engine*, *Godot* и *Bevy*.

2.3.1 *Unity*

Unity је један од најкоришћенијих комерцијалних погона у индустрији. Његова популарност произилази из релативно ниске улазне баријере, великог екосистема пакета, документације и мултиплатформске подршке.

Током већег дела свог развоја *Unity* је био заснован на моделу *GameObject + MonoBehaviour*, где су објекти сцене организовани кроз компоненте везане за хијерархију објеката. Са растом захтева за перформансама и већим бројем ентитета на сцени, *Unity* је увео *DOTS* (енг. *Data-Oriented Technology Stack*) који уводи *ЕКС* приступ и *data-oriented* дизајн [2].

2.3.2 *Unreal Engine*

Unreal Engine је такође један од најзначајнијих комерцијалних индустријских стандарда, посебно у сегменту комплексних тродимензионалних игара и AAA продукције.

Архитектура је заснована на *Gameplay Framework*-у где су учесник (енг. *Actor*) и компоненте основне јединице композиције [3]. Иако овај приступ није *ЕКС* у строгом смислу, он решава сличан проблем избегавања дубоких хијерархија наслеђивања кроз композицију функционалности.

2.3.3 *Godot*

Godot је погон отвореног кода који користи архитектуру засновану на чворовима (енг. *node-based architecture*). Сваки објекат у сцени представљен је чвором који може имати подређене чворове и специфичну функционалност.

Овакав приступ омогућава једноставно организовање логике сцене, али се архитектурално разликује од *ЕКС* приступа, јер је структура система оријентисана ка хијерархији чворова уместо ка обради хомогених скупова података.

2.3.4 *Bevy*

Bevy је савремени *Rust* погон отвореног кода чија је архитектура од почетка пројектована око *ЕКС* парадигме. У опису пројекта наглашава се да је реч о *data-oriented* погону заснованом на *ЕКС* моделу [4], [5].

Због директног ослањања на *Rust* екосистем и *data-oriented* приступ, *Bevy* представља значајну референтну тачку за радове који истражују развој погона за видео игре у програмском језику *Rust*.

2.4 Разлози избора програмског језика *Rust*

Циљ овог рада је доказати да ли је могуће реализовати функционалан *ЕКС* погон са рендеринг слојем без употребе небезбедног кода у апликативном слоју.

Из тог разлога је за израду погона изабран програмски језик *Rust*.

2.4.1 Меморијска безбедност

Rust обезбеђује меморијску безбедност кроз систем власништва (енг. *ownership*), позајмљивања (енг. *borrowing*) и животних векова (енг. *lifetimes*).

За разлику од многих системских језика, велики број грешака може бити откривен већ током компилације. То укључује:

- коришћење ослобођене меморије,
- двоструко ослобађање,
- невалидне мутабилне референце,
- део конкурентних грешака и *data race* ситуација.

Ове гаранције су посебно значајне у развоју погона за видео игре јер се велики број подсистема истовремено ослања на исте структуре података.

2.4.2 Поређење са C и C++ приступом

Традиционално су многи погони за видео игре развијани у програмском језику C или C++.

У C приступу програмер има потпуну контролу над меморијом, али је истовремено одговоран за исправност свих операција које се тичу управљања ресурсима.

Такав модел омогућава висок ниво флексибилности, али повећава ризик од грешака које је често тешко репродуковати и анализирати.

При развоју ЕКС система у C језику значајан део времена може бити потрошен на:

- праћење животног века података,
- дебаговање меморијских грешака,
- анализу корупције меморије,
- синхронизацију приступа дељеним структурама.

Rust помера велики део ових провера у фазу компилације. Последица тога је да се сложеност јавља раније у процесу развоја, током обликовања архитектуре, уместо касније током дебаговања.

За пројекат попут *Rusty* то представља значајну предност јер омогућава да се више времена посвети организацији ЕКС архитектуре и анализи перформанси, уместо отклањању меморијских проблема ниског нивоа.

2.4.3 Улога безбедног и небезбедног Rust-a

Rust није ограничен искључиво на безбедни подскуп језика. Поред механизма који омогућавају статичку проверу великог броја безбедносних својстава програма, језик пружа и могућност употребе небезбедног блока за операције чију исправност компајлер не може у потпуности да верификује.

Према званичној документацији језика, постојање механизма за небезбедне блокове кода произилази из чињенице да је статичка анализа по својој природи конзервативна, односно да не може увек са сигурношћу да докаже исправност свих легалних програма [6].

Употреба небезбедних блокова кода не подразумева искључивање правила језика, већ представља начин да програмер експлицитно преузме одговорност за одржавање одређених инваријанти које компајлер не може самостално да провери [6].

2.4.4 Могућност развоја погона без небезбедног кода

Циљ овог рада није доказивање да потпун спој графичких библиотека и погона може бити без небезбедног кода. Библиотеке нижег нивоа, попут графичких апстракција или интерфејса према оперативном систему, често користе небезбедне механизме.

Предмет анализе је апликативни и архитектурални слој погона.

Основна хипотеза рада гласи да је могуће развити:

- ЕКС архитектуру,
- систем компоненти,
- систем ентитета,
- организацију рендеринг слоја,
- управљање ресурсима,

без увођења небезбедних сегмената у доменској логици погона.

Практична реализација *Rusty* погона показује да је такав приступ изводљив за прост дводимензионални ЕКС погон, чији су приоритети јасна архитектура, модуларност, предвидиво управљање ресурсима и разумљив модел проширења функционалности.

Због тога је *Rust* представљао природан избор за овај рад, јер омогућава комбинацију системског нивоа контроле, модерних апстракција и меморијске безбедности без потребе за сакупљачем отпадака (енг. *garbage collector*).

Глава 3

Имплементација ЕКС језгра у Rusty погону

Ово поглавље детаљно описује како је ЕКС архитектура имплементирана у оквиру погона за видео игре Rusty.

За разлику од претходног поглавља, које даје теоријски и компаративни оквир ЕКС приступа, овде је фокус на конкретним структуралним одлукама у коду: организацији ентитета, складиштењу компоненти, догађајном механизму, извршавању система и интеграцији тих делова у централну структуру света (енг. *world*).

Код је организован модуларно и прати принцип раздвајања одговорности који је карактеристичан за *data-oriented* пројектовање [7].

3.1 Преглед датотека у Rusty погону

ЕКС језгро погона садржи се од следећих изворних датотека:

- `lib.rs` - улазна тачка погона, дефинише модуле и јавни АПИ;
- `entity.rs` - модел ентитета и управљање животним циклусом;
- `component.rs` - апстракције компоненти, складишта и менаџер компоненти;
- `event.rs` - догађајни модел и вишетипски догађајни редови;
- `system.rs` - дефиниција система и секвенцијални извршилац;
- `world.rs` - интеграциони слој који обједињује ентитете, компоненте и догађаје, уз могућност серијализације стања.

Ради провере исправности имплементације, сваки модул је засебно тестиран јединичним тестовима дефинисаним унутар одговарајуће изворне датотеке употребом атрибута `#[cfg(test)]`.

3.2 `lib.rs`: модулска декларација и јавни интерфејс

Датотека `lib.rs` прво излаже унутрашње модуле (`entity`, `component`, `event`, `world`, `system`, као и `render`), а затим реекспортије централне типове (`Entity`, `World`, `SystemExecutor` и друге), тако да корисник библиотеке може да ради са једноставним и конзистентним АПИ-јем.

```
pub mod entity;
pub mod component;
pub mod event;
pub mod world;
pub mod system;

pub use entity::{Entity, EntityManager};
pub use component::{Component, ComponentManager, HashMapComponentStorage};
pub use event::{Event, EventManager, EventQueue};
pub use world::World;
pub use system::{System, SystemExecutor};
```

Кључна предност реекспортовања централних типова кроз коренски модул библиотеке јесте стабилност јавног АПИ-ја. Кориснички код зависи од нпр. `rusty::World` или `rusty::Entity`, а не од дубоке интерне путање модула. Практично, то смањује трошак каснијих реорганизација изворног кода, јер унутрашње промене могу да остану неприметне за корисника ако се спољни АПИ не мења.

Алтернатива је била да се типови не реекпортују, већ да сваки корисник увози симболе из подмодула. Тај приступ је формално исправан, али води ка дужим use путањама и јачој спрези корисника са унутрашњом структуром библиотеке.

3.3 entity.rs: идентификатори, генерације и рециклажа ентитета

Модул entity.rs дефинише две централне структуре: Entity и EntityManager.

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord, Serialize, Deserialize)]
pub struct Entity {
    pub id: u32,
    pub generation: u32,
}
```

Entity је реализован као пар (id, generation), где је id идентификатор, а generation генерација тог идентификатора. Ово је познат образац за решавање проблема „висећих“ референци на обрисане ентитете у ЕКС системима [1].

EntityManager одржава:

- next_id за доделу нових идентификатора,
- free_ids као листу доступних идентификатора за поновну употребу,
- generations као вектор у коме сваки индекс директно одговара једном идентификатору, а вредност на тој позицији представља тренутну генерацију тог идентификатора.

```
#[derive(Serialize, Deserialize, Clone)]
pub struct EntityManager {
    next_id: u32,
    free_ids: Vec<u32>,
    generations: Vec<u32>,
}
```

create метода креира нови ентитет тако што прво покушава да рециклира доступан идентификатор из листе free_ids, а ако га нема, додељује нови идентификатор и иницијализује његову генерацију са вредношћу 0.

```
pub fn create(&mut self) -> Entity {
    if let Some(id) = self.free_ids.pop() {
        Entity { id, generation: self.generations[id as usize] }
    } else {
        let id = self.next_id;
        self.next_id += 1;
        self.generations.push(0);
        Entity { id, generation: 0 }
    }
}
```

destroy метода означава ентитет као неважећи тако што проверава да ли се генерација идентификатора поклапа са тренутном, затим повећава генерацију идентификатора и ставља id у free_ids ради поновне употребе.

```
pub fn destroy(&mut self, entity: Entity) {
    if (entity.id as usize) < self.generations.len() && self.generations[entity.id as usize]
    == entity.generation
    {
        self.generations[entity.id as usize] += 1;
        self.free_ids.push(entity.id);
    }
}
```

3.3.1 Зашто не забранити рециклажу идентификатора?

Потпуна забрана поновне употребе идентификатора делује безбедно, али у пракси доводи до исцрпљивања свих могућих идентификатора у дуготрајним симулацијама са великим бројем ентитета. Уместо тога, рециклажа идентификатора уз употребу генерација пружа ограничен и стабилан простор идентификатора, уз задржавање безбедности референци.

3.3.2 Зашто генерације, а не само идентификатори?

Када би ентитет био моделован само као `id: u32` без генерације, појавио би се проблем рециклирања идентификатора:

1. ентитет са `id = 5` се обрише,
2. исти `id = 5` се касније додели новом ентитету,
3. стари референцирани ентитет и даље постоји у системима и сада погрешно указује на нови ентитет.

Комбинација идентификатора и генерације решава овај проблем тако што сваки животни циклус истог идентификатора добија нову верзију. Стари (5, 0) и нови (5, 1) више нису упоредиви као исти ентитет, што омогућава детерминистичку проверу валидности референци.

3.3.3 Зашто не један u64?

Иако би се идентификатор и генерација могли спаковати у једну 64-битну вредност, овај приступ у ЕКС архитектурама има неколико недостатака. Пре свега, он уклања експлицитну структурну разлику између идентификатора и верзије, што смањује читљивост и отежава логичко расуђивање о систему.

Поред тога, одвојено чување `id` и `generation` често омогућава боље кеш понашање у *data-oriented* дизајну, јер системи могу независно да обрађују идентификаторе и метаподатке о верзијама.

3.3.4 Зашто не UUID?

Коришћење *UUID* (енг. *Universally Unique Identifier*) вредности обезбеђује глобалну јединственост, али за прост ЕКС систем уводи значајан меморијски и перформансни трошак. *UUID* обично заузима 128 бита, што је четири пута више од 32-битног идентификатора, уз додатно погоршање кеш локалности приликом масовне обраде ентитета.

UUID је погоднији за дистрибуиране системе и перзистентне ресурсе, али мање ефикасан у високофреквентним симулацијама са великим бројем краткоживећих објеката.

3.3.5 Инжењерски компромис

У пракси, модел заснован на пару `id` и `generation` представља компромис између перформанси, меморијске ефикасности и безбедности. Он омогућава:

- $O(1)$ приступ ентитетима,
- ефикасно коришћење меморијског простора,
- поуздано откривање референци на обрисане ентитете.

Овај приступ је широко прихваћен у савременим ЕКС системима, попут *Bevy* [8].

3.4 `component.rs`: типизоване компоненте преко *type erasure* слоја

`component.rs` је најобимнији део погона и садржи комплетну инфраструктуру за складиштење компоненти различитих типова. Сваки тип компоненте има своје засебно складиште, чиме се обезбеђује типска изолација података и избегава мешање различитих структура у меморији.

3.4.1 *Trait Component*

`Component` је *marker trait* који дефинише скуп ограничења која свака компонента мора да испуњава. Овај *trait* представља заједнички именитељ који омогућава да сва различита складишта компоненти буду третирана на унифициран начин у оквиру система.

Конкретно, компонента мора да:

- имплементира Any, чиме се омогућава идентификација типа у време извршавања,
- има 'static животног века, што омогућава безбедно коришћење кроз механизме као што је Any.
- подржава серијализацију и десеријализацију путем Serialize и Deserialize,

Ова ограничења омогућавају да се типови компоненти динамички препознају током извршавања програма (коришћењем TypeId или Any), уместо да се ослањају искључиво на статичку типску проверу у време компајлирања.

```
pub trait Component: Any + Serialize + for<'de> Deserialize<'de> + 'static {}
```

```
impl<T: Any + Serialize + for<'de> Deserialize<'de> + 'static> Component for T {}
```

Ограничење на Serialize / Deserialize није универзални захтев свих EKC библиотека, али је у Rusty уведено јер World подржава чување и читавање стања у JSON (енг. *JavaScript Object Notation*) формату преко serde радног оквира.

Пошто сваки тип компоненте има своје засебно складиште, јавља се потреба да се та хетерогеност сакрије иза јединственог интерфејса.

3.4.2 Брисање типа складишта компоненти и динамички диспеч

У Rust-у, типски систем по правилу захтева да конкретни типови буду познати у време компајлирања. Међутим, у EKC архитектури, ComponentManager у једној структури управља складиштима различитих компоненти (нпр. Position, Velocity, Health и сл.), при чему свака од њих има различит тип.

Да би се ово омогућило, користи се техника брисања типа (енг. *type erasure*), где се конкретни типови „скривају“ иза заједничког интерфејса dyn ComponentStorage. На тај начин, различита складишта могу да се чувају у истој колекцији без познавања њиховог конкретног типа у тренутку компајлирања.

Овако добијени апстрактни слој омогућава да се сва складишта индексирају по TypeId, чиме се формира централни регистар свих компоненти у систему.

Конкретан тип се поново „открива“ тек када је потребно, коришћењем механизма downcast-a (downcast_ref, downcast_mut). Ова операција представља контролисано враћање са динамичког типа на конкретан тип, али само у случају када је стварни тип у складу са очекиваним.

```
pub trait ComponentStorage: Any {  
    fn as_any(&self) -> &dyn Any;  
    fn as_any_mut(&mut self) -> &mut dyn Any;  
    fn remove(&mut self, entity: Entity);  
}
```

У наставку се види пример враћања конкретног складишта за одређени тип T:

```
pub fn get_storage_by_type<T: Component>(&self) -> Option<&HashMapComponentStorage<T>> {  
    self.storages  
        .get(&TypeId::of::<T>())?  
        .as_any()  
        .downcast_ref::<HashMapComponentStorage<T>>()  
}
```

Другим речима, јавни интерфејс остаје потпуно статички типски безбедан (нпр. get_component::<T>), док се унутрашња имплементација ослања на динамички диспеч (енг. *dynamic dispatch*) како би се омогућила хетерогена колекција складишта у једном систему.

Иако је приступ складиштима унификован, унутрашња репрезентација сваког складишта остаје специфична за конкретан тип компоненте.

3.4.3 HashMapComponentStorage<T: Component> и ComponentManager

Конкретна имплементација складишта је HashMapComponentStorage<T>, где је унутрашња структура HashMap<Entity, T>. Зато је приступ „ентитет -> компонента“ просечно $O(1)$, уз очекиване карактеристике хеш структура.

ComponentManager има два речника:

- `storages: HashMap<TypeId, Box<dyn ComponentStorage>>`,
- `type_names: HashMap<String, TypeId>`.

Први речник служи за приступ складиштима по типу током извршавања кода, док други уводи стабилно текстуално име типа за серијализацију. Ово је важно јер сам TypeId није погодан као интероперабилан формат за трајно чување података.

Главне операције менаџера су:

- `register<T>(name)` - региструје тип компоненте по називу и прави складиште за тај тип ако не постоји,
- `add_component` - додаје компоненту уз креирање складишта по потреби,
- `get_storage_by_type` и `get_storage_mut_by_type` - типизован приступ уз *downcast*,
- `remove_all_components` - уклања везе између свих компоненти и датог ентитета,
- `serialize_all` и `deserialize_all` - серијализација и десеријализација свих регистрованих складишта компоненти.

Овај модел имплицира да је целокупан систем организован као скуп независних речника који се централизовано управљају преко менаџера компоненти.

3.4.4 Избор HashMap модела за складишта компоненти и његове алтернативе

Предности HashMap приступа за чување свих типова складишта компоненти у Rusty су:

- врло једноставно додавање / брисање компоненте,
- директно адресирање преко Entity идентификатора,
- мала сложеност кода ради лакшег разумевања и проширивања,
- лакша серијализација речника у JSON.

Мане произилазе из природе речника:

- лошија кеш локалност код масовног проласка кроз компоненте,
- додатни трошак хеш рачунања,
- мање погодан модел за SIMD (енг. *Single Instruction, Multiple Data*) / колонску обраду великих скупова података.

3.4.4.1 Архетипски приступ

Архетипски ЕКС организује ентитете у „табеле“ према тачној комбинацији компоненти коју поседују. Ако један ентитет има (Position, Velocity), а други (Position, Health), они припадају различитим архетипима.

Практична последица је да се компоненте чувају као колоне унутар архетипа, па су итерације система који обрађују фиксну комбинацију компоненти често веома брзе због добре кеш локалности [7].

Цена тог приступа је „миграција“ ентитета приликом додавања или брисања компоненте:

- промена сета компоненти мења архетип ентитета,
- ентитет мора да се премести у другу табелу,
- потребно је ажурирати мапирања и индексе.

Дакле, архетипски модел је веома добар за сценарије са честим итерацијама над групама ентитета који су везани за исте компоненте, али је имплементационо сложенији од почетног HashMap решења.

3.4.4.2 *Sparse set* организација

Sparse set је компромис између једноставних мапа и строго архетипских табела. Класична варијанта има:

- `dense`: компактну листу активних ентитета,
- `sparse`: речник `entity_id` -> индекс унутар `dense`,
- паралелан низ компоненти који прати `dense` редослед.

Тиме се добијају битне особине:

- брза провера постојања компоненте,
- компактна итерација по постојећим компонентама,
- релативно једноставно брисање (обично *swap-remove* техника).

У односу на архетип, *sparse set* је једноставнији за имплементацију и често веома добар у пракси. У односу на `HashMap`, углавном има бољу локалност при итерацији, али захтева више пажње око индекса и конзистентности структуре.

3.4.4.3 *SoA* колоне и хибридни модели

SoA (енг. *Structure of Arrays*) чува свако поље у засебној колони (нпр. све `x` координате заједно), што додатно побољшава секвенцијално читање и *SIMD* обраду. Међутим, *SoA* захтева пажљив дизајн АПИ-ја и често утиче на ергономију кода.

У индустрији су чести хибриди: архетип за често коришћене компоненте и ређи, динамични подаци у спољним структурама. `HashMap` као полазна тачка је оправдана јер чини изборе експлицитним, а каснију миграцију ка комплекснијем складишту могућом без рушења целог АПИ-ја.

3.5 `event.rs`: систем догађаја са редовима по типу

Модул `event.rs` уводи механизам асинхроне комуникације између система путем догађаја.

`Event` је *marker trait* (`Any + 'static`), док `EventQueue<E>` представља *FIFO* (енг. *first in, first out*) ред конкретног типа догађаја заснован на `VecDeque<E>`.

```
pub struct EventQueue<E: Event> {
    events: VecDeque<E>,
}

pub fn push<E: Event>(&mut self, event: E) {
    self.register:::<E>();
    if let Some(queue) = self.get_queue_mut:::<E>() {
        queue.push(event);
    }
}
```

За складиштење више типова редова у једном менаџеру користе се *trait* `EventQueueTrait` и речник `HashMap<TypeId, Box<dyn EventQueueTrait>>` унутар `EventManager` структуре. Као код компоненти, гарантује се статичка типска безбедност на АПИ слоју и динамички *type erasure* унутар менаџера.

3.6 `system.rs`: системи и извршни редослед

`system.rs` дефинише *trait* `System` са методом `run(&mut self, world: &mut World)`, и извршилац `SystemExecutor` који чува `Vec<Box<dyn System>>`.

```
pub trait System {
    fn run(&mut self, world: &mut World);
}

pub struct SystemExecutor {
    systems: Vec<Box<dyn System>>,
}
```



```

}

pub fn run(&mut self, world: &mut World) {
    for system in &mut self.systems {
        system.run(world);
    }
}

```

Извршавање је секвенцијално, редом којим су системи додати у вектор система, чиме се добија једноставан и детерминистички модел извршавања. Тестови у модулу експлицитно показују да редослед утиче на резултат (нпр. увећавање пре удвостручавања вредности није исто што и удвостручавање пре увећавања вредности).

У литератури и индустријским ЕКС системима често је и паралелно извршавање, где се системи извршавају конкурентно ако нема конфликтног приступа подацима. Такав модел може да повећа искоришћење вишејезгарних процесора, али захтева значајно сложеније праћење зависности писања и брисања података и сложенији АПИ.

3.7 world.rs: интеграциони слој ЕКС језгра погона

World је централна фасада језгра и садржи сва три менаџера:

- EntityManager,
- ComponentManager,
- EventManager.

World игра излаже компактни АПИ и од корисника Rusty погона сакрива комплексност унутрашњих подсистема.

```

pub fn destroy_entity(&mut self, entity: Entity) {
    self.components.remove_all_components(entity);
    self.entities.destroy(entity);
}

pub fn take_events<E: Event>(&mut self) -> Vec<E> {
    let mut events = Vec::new();
    if let Some(queue) = self.events.get_queue_mut:::<E>() {
        while let Some(event) = queue.pop() {
            events.push(event);
        }
    }
    events
}

```

3.7.1 Животни циклус ентитета у World-y

Метода destroy_entity прво позива remove_all_components, па тек онда entities.destroy(entity). Овај редослед је важан јер спречава да у складиштима остану сирочад (енг. *orphan*) компоненте за ентитет који више не постоји. У комбинацији са генерацијским моделом из entity.rs, добија се конзистентан механизам брисања и поновне употребе идентификатора без логичке колизије старих и нових ентитета.

3.7.2 Серијализација и десеријализација света

World дефинише интерну структуру WorldSaveData са кључним пољима:

- стање менаџера ентитета,
- серијализована складишта компоненти.

Чување (save) и учитавање (load) реализовани су у JSON формату преко serde_json.

Након учитавања се позива `events.clear()`, чиме се избегава да се стари догађаји из претходног извршавања пренесу у ново учитано стање. Овиме стање света остаје поновљиво, а догађаји задржавају улогу транзијентног механизма комуникације, уместо да постану део трајног стања света.

3.8 Пример примене: текстуална борбена игра

Да би се видело како описана архитектура функционише у стварној употреби, развијена је текстуална игра у којој се *Rusty ЕКС* користи за једноставну потезну борбу играча против више непријатеља:

- ентитети представљају учеснике у игри (играч и непријатељи),
- компоненте носе податке (`Name`, `Health`, `Damage`, `Defending`, маркери `Player` и `Enemy`),
- догађаји (`AttackEvent`) представљају намеру акције,
- систем (`DamageSystem`) претвара догађаје у конкретну промену стања,
- `SystemExecutor` контролише редослед извршавања,
- `World` служи као фасадни АПИ кроз који се све оркестрира.

Следећи исечак кода показује почетну конфигурацију света и регистрацију система:

```
let mut world = World::new();

let player = world.create_entity();
world.add_component(player, Name("Hero".to_string()));
world.add_component(player, Player);
world.add_component(player, Health { hp: 45, max: 45 });
world.add_component(player, Damage { value: 7 });
world.add_component(player, Defending(false));

let mut executor = SystemExecutor::new();
executor.add_system(DamageSystem);
```

Овде се јасно види практична улога `World` АПИ-ја: `create_entity` враћа ентитет, док се `add_component` позива више пута да би се формирао комплетан „профил“ тог ентитета. Другим речима, понашање није везано за класну хијерархију, већ настаје композицијом компоненти.

Ток једног потеза реализован је кроз догађај и накнадно извршавање система:

```
world.push_event(AttackEvent {
    attacker: player,
    target: enemy,
    damage: dmg,
});

executor.run(&mut world);
```

Позив `push_event` смешта напад у ред догађаја, а тек у `executor.run` систем `DamageSystem` преузима све `AttackEvent` догађаје преко `take_events` и примењује последице на `Health` циљаног ентитета напада. Тиме се раздваја опис самог догађаја од промене стања над циљем, у овом случају `Health` компонентом. Догађај само бележи да је дошло до напада, док се конкретна измена животних поена играча или непријатеља извршава накнадно у оквиру система који обрађују тај догађај.

Унутар система се користе `get_component` и `get_component_mut`: први за читање тренутног стања (име, улога, вредност напада), а други за безбедно ажурирање променљивих података (смањење `Health`). Тако библиотека задржава јасну границу између мутабилног и имутабилног приступа подацима у оквиру једног кадра извршавања.

Компонента `Name` је минимална *tuple struct* дефиниција (`struct Name(String);`) која служи за приказ у корисничком интерфејсу (енг. *UI, user interface*), без утицаја на саму механику борбе.

Насупрот томе, `Player` је маркер компонента без поља (`struct Player;`). Она је сигнал системима да је реч о ентитету који представља играча. Та разлика је јасна и у `DamageSystem`: `world.get_component::<Player>(attack.attacker).is_some()` одређује да ли се исписује порука у другом лицу („You strike...“) или у трећем („X hits you...“).

И непријатељи се уводе композиционо, кроз исти АПИ, без посебне класе. Уместо класе *Enemy*, у коду постоји скуп параметара (`enemy_attributes`) из ког се у петљи креира више ентитета:

```
let enemy_attributes = vec![
    ("Goblin", 12, 3, vec!["Slash", "Bite"]),
    ("Orc", 18, 5, vec!["Heavy Swing", "Headbutt"]),
    ("Necromancer", 22, 6, vec!["Shadow Bolt", "Bone Spike"]),
];

for (name, hp, dmg, _attacks) in &enemy_attributes {
    let e = world.create_entity();
    world.add_component(e, Name(name.to_string()));
    world.add_component(e, Enemy);
    world.add_component(e, Health { hp: *hp, max: *hp });
    world.add_component(e, Damage { value: *dmg });
    enemy_entities.push(e);
}
```

Оваква конструкција непријатеља добро илуструје основну ЕКС предност: нов тип противника не захтева нову хијерархију типова, већ нову комбинацију компоненти и параметара. Компонента *Enemy* овде, као и *Player*, има улогу маркера који омогућава филтрирање и другачије понашање у логици игре.

Са становишта корисника библиотеке, логика игре се дефинише у једној `loop` петљи у функцији `main`, у оквиру које се ручно описује редослед интеракција (акција играча, акција непријатеља и покретање система). Са становишта *Rusty* погона, свака фаза ове петље само активира одговарајуће механизме обраде догађаја и система.

Поједностављено, ток изгледа овако:

```
loop {
    // 1) услови завршетка
    // 2) читање акције играча: attack / defend / quit
    // 3) push_event(...) за напад и executor.run(&mut world)
    // 4) ако је непријатељ жив, он шаље AttackEvent ка играчу
    // 5) system_executor.run(&mut world) за непријатељски потез
}
```

Циклус игре (енг. *game loop*) не садржи логику умањења *HP*-а, већ само оркестрацију догађаја и позив система. Због тога је ток игре лако проширив: увођење нове акције, статусних ефеката или *AI* правила углавном се своди на додавање нових догађаја и / или система који их обрађују, док основна структура циклуса игре и модел оркестрације остају непромењени.

Глава 4

Опције за рендеринг слој у *Rusty* погону

Да би се перформансе и понашање *ЕКС* језгра могле испитати у условима који одговарају стварној употреби, потребно је имплементирати и једноставан рендеринг слој. Иако рендеринг није у фокусу овог рада, његово постојање омогућава изградњу потпуно функционалне дводимензионалне игре која ће служити као окружење за симулацију и стрес тестирање развијеног *ЕКС* система. На тај начин могуће је посматрати понашање језгра у сценаријима који укључују велики број ентитета, честе измене компоненти и континуирано извршавање система током рендеровања кадрова.

Из тог разлога потребно је одабрати библиотеку или апстракциони слој за рендеринг који ће се уклопити у архитектуру погона и омогућити реализацију тестних сценарија без увођења непотребних ограничења. Изабрано решење треба да задовољи следеће захтеве:

- да не намеће сопствени циклус игре,
- да омогући ефикасно приказивање великог броја спрајтова и текстура, како би се могла симулирати оптерећења карактеристична за стварне игре,
- да подржава прилагођавање рендеринг процеса ради лакшег повезивања са *ЕКС* архитектуром,
- да буде мултиплатформско решење доступно на оперативним системима *Windows*, *macOS* и *Linux*,
- да омогући писање апликативног слоја у безбедном *Rust* коду.

У оквиру *Rust* екосистема могу се издвојити три основна приступа за реализацију овог слоја:

- директно коришћење графичких *АПИ*-ја ниског нивоа (*wgpu*, *ash*, *vulkano*),
- апстракционе библиотеке засноване на постојећим *АПИ*-јима (*pixels*, *softbuffer*),
- наменске библиотеке за дводимензионалне игре (*macroquad*, *ggez*, *raylib*),

4.1 Графичке библиотеке ниског нивоа

4.1.1 *ash*

ash је *Rust* омотач (енг. *wrapper*) око *Vulkan* графичког *АПИ*-ја [9]. Библиотека пружа готово директан приступ *Vulkan* функцијама без значајних апстракција изнад самог *АПИ*-ја.

ash је погодан за ситуације у којима програмер жели потпуну контролу над свим аспектима *Vulkan* извршног окружења. Ова карактеристика га чини моћним, али и захтевним: за елементарно цртање троугла потребно је ручно иницијализовати инстанцу, физички и логички уређај, командне редове, пролаз за рендеровање (енг. *render pass*), графички пајплајн (енг. *pipeline*), бафере за слике и синхронизационе примитиве.

Додатно, *ash* нема подршку за *Windows*, *macOS* и *Linux* кроз јединствени апстракциони слој на нивоу библиотеке. Мултиплатформска подршка захтева ручно руковање разликама између платформи и интеграцију са библиотеком попут *winit* за управљање прозорима.

Главна мана *ash*-а у контексту *Rusty* погона јесте недостатак апстракције јер сваки детаљ *Vulkan* инфраструктуре постаје одговорност програмера.

4.1.2 *vulkano*

vulkano је *Rust* библиотека која уводи слој апстракције изнад *Vulkan* *АПИ*-ја [10]. За разлику од *ash*-а, *vulkano* аутоматски управља делом синхронизационих и меморијских детаља, нудећи апстракцију ресурса са јаснијим моделом власништва у духу *Rust*-а.

vulkano генерише типски безбедне омотаче за *Vulkan* структуре током компајлирања, чиме се одређена класа грешака у употреби *Vulkan* ресурса може открити пре извршавања програма.

Ипак, *vulkano* остаје везан искључиво за *Vulkan*, па нема изворне подршке за *macOS* (где је доступан само *Metal* преко *MoltenVK* слоја), нити за *WebAssembly*. Мултиплатформска компатибилност захтева додатну конфигурацију и ослањање на средства трећих лица.

Са становишта погона за дводимензионалне игре, *vulkano* и даље уводи знатну сложеност приликом постављања иницијалне инфраструктуре.

4.2 Апстракционе библиотеке

4.2.1 *pixels*

pixels је библиотека за хардверски убрзан рендеринг пиксел-бафера (енг. *pixel buffer*) у *Rust*-у [11]. Основна идеја је да програмер директно пише пикселе у меморијски бафер у *RGBA* формату, а *pixels* тај бафер ефикасно преноси на екран преко *wgpu*.

Употреба је намерно поједностављена: *pixels* ствара прозор, управља површином за цртање (енг. *surface*) и брине о размеривању (енг. *scaling*) пиксел-бафера на стварну резолуцију прозора.

```
let mut pixels = {
    let surface_texture = SurfaceTexture::new(WIDTH, HEIGHT, &window);
    Pixels::new(WIDTH, HEIGHT, surface_texture)?
};

let frame = pixels.frame_mut();
for pixel in frame.chunks_exact_mut(4) {
    pixel.copy_from_slice(&[0x48, 0xb2, 0xe8, 0xff]);
}
pixels.render()?;
```

pixels је одличан за прототипове, демо апликације и игре у стилу ретро пиксел уметности где се графика генерише потпуно процесорски, без употребе текстура и шејдера.

Ограничење *pixels* библиотеке за потребе *Rusty* погона је архитектурална природа саме библиотеке. Она не нуди механизме за управљање текстурама, спрајтовима, беч рендерингом (енг. *batching*) нити за прилагођене шејдере. Сваки елемент игре морао би ручно да се растеризује у пиксел-бафер, без хардверске подршке за операције попут трансформација и блендовања.

4.2.2 *softbuffer*

softbuffer је библиотека за директно цртање пиксела у прозор без убрзања графичким процесором [12]. Она апстрахује разлике између платформских АПИ-ја (нпр. *Win32*, *X11*, *Wayland*, *CoreGraphics*) и излаже јединствен интерфејс за писање у фрејм бафер (енг. *framebuffer*).

softbuffer је намењен искључиво процесорском рендерингу и нема никакву интеграцију са графичким процесором. За потребе *Rusty* погона, где је убрзање графичким процесором важно чак и у дводимензионалном контексту, *softbuffer* није одговарајући избор.

4.3 Наменске библиотеке за дводимензионалне игре

4.3.1 *ggez*

ggez је *Rust* библиотека инспирисана *LÖVE* радним оквиром за *Lua* [13]. Циљ библиотеке је да пружи минималан и прагматичан скуп алата за дводимензионалне игре без уводног трошка постављања графичке инфраструктуре ниског нивоа.

ggez нуди:

- цртање спрајтова и геометријских примитива,
- управљање звуком,
- улазне догађаје,
- управљање ресурсима путем система датотека.

Унутар *ggez* библиотеке рендеринг је реализован преко *wgpu*, али ова чињеница остаје скривена иза апстракционог слоја библиотеке.

Кључно ограничење *ggez* библиотеке за потребе *Rusty* погона је чврста спреженост петље игре са унутрашњим догађајима библиотеке. Кориснички код мора да прати унапред одређени образац постављања и ажурирања (нпр. имплементација *EventHandler trait*-а), чиме се смањује могућност слободне интеграције са спољним *ЕКС* слојем.

4.3.2 *macroquad*

macroquad је минималистичка библиотека за дводимензионалне игре у *Rust*-у [14]. Дизајнирана је са циљем да смањи улазну баријеру за развој игара: не захтева спољну петљу ни сложену иницијализацију, а основни приказ на екрану може се остварити у неколико линија кода.

```
use macroquad::prelude::*;

#[macroquad::main("Game")]
async fn main() {
    loop {
        clear_background(BLACK);
        draw_circle(200.0, 200.0, 30.0, YELLOW);
        next_frame().await;
    }
}
```

macroquad подржава *Windows*, *macOS*, *Linux* и *WebAssembly*, а нуди и функционалности попут цртања текстура, текста, геометрије, управљања улазом и репродукције звука.

Унутрашња имплементација *macroquad* библиотеке заснована је на *OpenGL* (преко *miniquad* апстракционог слоја), а не на савременим графичким АПИ-јима попут *Vulkan*, *Metal* или *DirectX 12*.

Ово је значајно ограничење за пројекте усмерене ка дугорочној компатибилности и потпуном искоришћавању савремених графичких функционалности. *OpenGL* је у статусу одржавања на *macOS* (почев од верзије 10.14, *Apple* означава *OpenGL* као застарели АПИ), а *WebGL* (на ком *macroquad* заснива веб подршку) нема директан еквивалент у *WebGPU* стандарду.

Са архитектуралног становишта, *macroquad* ствара потешкоће при интеграцији са засебним *ЕКС* слојем јер петља извршавања и управљање стањем постоје унутар библиотеке, умешани са логиком рендеровања.

4.3.3 *raylib-rs*

raylib је популарна *C* библиотека за развој игара, а *raylib-rs* представља *Rust* омотач преко *FFI* (енг. *Foreign Function Interface*) слоја [15].

raylib пружа широк скуп функционалности: цртање дводимензионалних и тродимензионалних примитива, управљање улазом, звуком и прозором.

Ограничење *raylib-rs* за потребе *Rusty* погона произилази из природе *FFI* слоја: корисник мора да ради са небезбедним кодом на граничном слоју преко ког се комуницира са *C* кодом. Ово директно противречи основној хипотези рада, која се заснива на показивању да је могуће реализовати апликативни слој погона без употребе небезбедног кода.

4.4 Аргументација одабира *wgpu*

wgpu је Rust имплементација WebGPU стандарда која унутрашњим слојем апстракције подржава *Vulkan*, *Metal*, *DirectX 12*, *DirectX 11*, *OpenGL* и *WebAssembly* преко WebGPU [16].

Кључне предности *wgpu* у односу на претходно споменуте опције за имплементацију рендеринг слоја у Rusty су:

4.4.1 Мултиплатформска подршка без небезбедног кода у апликативном слоју

wgpu пружа јединствен АПИ за рендеринг на свим значајним платформама, без потребе да апликативни слој директно комуницира са платформски специфичним графичким АПИ-јем. Унутрашњост *wgpu* библиотеке користи небезбедне блокове тамо где је то неопходно (нпр. у интерфејсима према *Vulkan* или *Metal*), али та небезбедност остаје енкапсулирана унутар библиотеке и не продире у апликативни слој погона.

Ово је директно у складу са хипотезом рада: апликативни и архитектурални слој Rusty погона може бити написан у потпуности у безбедном Rust коду, ослањајући се на то да *wgpu* интерно управља небезбедним кодом.

4.4.2 Директна интеграција са Rust власничким моделом

wgpu АПИ је пројектован у складу са Rust системом власништва и позајмљивања. Ресурси попут бафера, текстура и командних кодера управљају се преко власничких структура са јасно дефинисаним животним циклусом. На тај начин, неисправна употреба ресурса може бити откривена у фази компајлирања уместо у тренутку извршавања програма.

4.4.3 Прилагодљивост рендеринг тока

За разлику од библиотека попут *macroquad* и *ggez*, *wgpu* не намеће структуру циклуса игре нити унапред одређен скуп апстракција за спрајтове или сцену. Програмер потпуно контролише организацију рендеринг тока: структуру пролаза рендеровања, формат шејдера, распоред везивних група (енг. *bind groups*) и редослед извршавања команди.

Ова прилагодљивост је важна за интеграцију са ЕКС архитектуром, јер рендеринг системи у Rusty погону могу директно управљати ресурсима графичког процесора, без посредних апстракција које би наметале властиту организацију сцене.

4.4.4 Активно одржавање и широка употреба у екосистему

wgpu је библиотека коју активно одржава тим *Mozilla Foundation* и *wgpu* заједнице. Она представља основу рендеринг слоја у *Firefox* прегледачу (за WebGPU имплементацију), у *Bevy* погону и у бројним другим пројектима у Rust екосистему.

Глава 5

Имплементација рендеринг слоја уз *wgpu*

Након дефинисања и имплементације основног *EKS* језгра у оквиру *Rusty* погона, неопходно је омогућити визуелизацију стања света ради лакшег праћења симулација и развоја видео игара. У овом поглављу описан је поступак имплементације рендеринг слоја који користи *wgpu* библиотеку за хардверски убрзано исцртавање дводимензионалних елемената.

5.1 Архитектура рендеринг слоја

Главна компонента рендеринг система је структура *Renderer2D*, која служи као омотач око *wgpu* графичког контекста. Она је одговорна за управљање животним циклусом графичког уређаја, креирање и ажурирање *wgpu* ресурса (као што су бафери и текстуре) и извршавање команди за цртање.

Иницијализација рендерера је комплексан процес који захтева координацију неколико кључних *wgpu* објеката:

- **Instance**: почетна тачка, обезбеђује приступ графичким адаптерима.
- **Adapter**: представља физички графички уређај (нпр. графичка картица).
- **Device** и **Queue**: **Device** је логички уређај који омогућава креирање ресурса, док **Queue** служи за слање команди графичком процесору.
- **Surface** и **Swapchain**: дефинишу прозорски контекст и механизам за двоструко баферовање (енг. *double buffering*).

Функција `new_async` асинхронно иницијализује ове ресурсе, прилагођавајући се доступном графичком АПИ-ју (*Vulkan*, *Metal*, *DirectX 12*) на домаћинском оперативном систему. Ова апстракција је кључна за преносивост и перформансе.

5.2 Основе *wgpu* ресурсног модела

wgpu захтева експлицитно управљање ресурсима. У оквиру *Rusty* погона, рендеринг је заснован на неколико основних градивних блокова:

5.2.1 Вертексни и индексни бафери

Спрајтови се представљају као правоугаоници (квадови). Дефинисани су помоћу `QUAD_VERTICES` (позиције темена и текстурне координате) и `QUAD_INDICES` (индекси за рендеровање троуглова). Ови подаци се преносе на меморију графичког процесора путем бафера, што омогућава ефикасно исцртавање стотина спрајтова у једном позиву.

5.2.2 Рендеринг пајплајн

Рендеринг пајплајн дефинише стање графичког процесора. Он укључује:

- **Шејдер модул**: скуп *WGS*L инструкција за обраду темена и фрагмената.
- **PipelineLayout**: дефинише како су ресурси (униформе, текстуре) мапирани у шејдере.
- **PrimitiveState**: конфигурација за растеризовање (нпр. начин слагања троуглова).

5.2.3 Везивне групе

Оне омогућавају шејдерима приступ подацима као што су текстуре (*TextureView*), сэмплери (*Sampler*) и униформни бафери (нпр. матрице камере). Користећи везивне групе, *Rusty* ефикасно преноси податке између процесора и графичког процесора.

5.3 Систем ресурса и текстура

Управљање текстурама је реализовано путем `TextureId` идентификатора који служи као апстрактна референца. `Renderer2D` одржава `HashMap<TextureId, TextureResource>` који повезује идентификаторе са стварним графичким ресурсима у меморији.

Учитавање текстура користи `image` библиотеку за декодирање података са диска у *RGBA* формат, након чега се подаци преносе на графички процесор помоћу `wgpu::util::DeviceExt::create_texture_with_data`. Овај систем омогућава динамичко додавање текстура током рада погона, без потребе за поновним покретањем графичког пајплајна.

5.4 Интеграција са ЕКС компонентама

Повезивање рендеринг слоја са ЕКС језгром остварено је дефинисањем специјализованих компоненти које садрже податке потребне за исцртавање:

- `Transform2D`: чува информације о позицији, ротацији и размери ентитета у простору сцене.
- `SpriteComponent`: дефинише визуелне карактеристике ентитета, укључујући идентификатор текстуре, изворни правоугаоник за мапирање текстуре (енг. *texture atlas*), величину приказа, слој исцртавања (енг. *z-index*) и боју (енг. *tint*).

Ове компоненте се обрађују током извршавања `RenderSystem`-а, који представља спону између података сцене и графичког уређаја.

5.5 RenderSystem и петља рендеровања

`RenderSystem` је стандардни ЕКС систем који се извршава унутар циклуса игре. Његова улога је да приступи свим ентитетима који поседују и `Transform2D` и `SpriteComponent` и споји њихове податке у облик погодан за графички процесор и проследи их рендереру.

Унутар `RenderSystem::run` методе, врши се закључавање инстанце `Renderer2D` помоћу `Arc<Mutex<Renderer2D>>`, чиме се осигурава безбедан приступ графичким ресурсима из вишенитног окружења:

```
impl System for RenderSystem {
    fn run(&mut self, world: &mut World) {
        if let Ok(mut renderer) = self.renderer.lock() {
            renderer.render_world(world);
        }
    }
}
```

Метода `render_world` прикупља видљиве спрајтове, сортира их према слоју исцртавања како би се обезбедило правилно слагање слојева, и на крају позива функцију `render_batch_with_order`. Ова функција групише спрајтове са истим текстурама како би се смањио број *draw* позива, што је кључна оптимизација за перформансе.

5.6 Употреба wgpu за графичко убрзање

wgpu је одабран јер пружа апстракцију високог нивоа која задржава перформансе графичких АПИ-ја ниског нивоа. У оквиру *Rusty* рендерера, *wgpu* омогућава коришћење сопствених шејдера писаних у *WGLSL* језику (енг. *WebGPU Shading Language*).

Шејдер `sprite.wgsl` извршава следеће операције:

1. Вертексни шејдер преузима податке о трансформацији (позиција, ротација, скала) из инстансних бафера и претвара их у *Clip Space* координате, користећи матрицу пројекције камере.
2. Фрагментни шејдер узоркује (енг. *sampling*) текстуру, примењује боју и врши алфа-блендовање пиксела.

Овакав дизајн омогућава да се већина тешких операција (нпр. матричне трансформације, апроксимација боја) извршава на графичком процесору, док *ЕКС* језгро остаје растерећено и фокусирано на управљање логиком игре и стањем света.

Глава 6

Евалуациона апликација и тестирање перформанси

Ради објективне евалуације перформанси *Rusty* погона и његовог *ЕКС* језгра, развијена је наменска евалуациона апликација. Ова апликација служи као визуелни стрес тест који омогућава посматрање понашања система у условима високог оптерећења, користећи искључиво функционалности самог *Rusty* погона без ослањања на спољне графичке библиотеке или друга готова решења.

6.1 Функционалности евалуационе апликације

Евалуација се заснива на симулацији великог броја ентитета (квадрата) који се независно крећу унутар прозора. Апликација омогућава кориснику интерактивну контролу над параметрима симулације у реалном времену:

- Динамичко скалирање броја ентитета: Корисник може повећавати или смањивати број спрајтова на екрану, чиме се директно мења оптерећење *ЕКС* система и број *draw* позива.
- Контрола брзине кретања: Могуће је убрзати или успорити кретање ентитета, што утиче на учесталост ажурирања *Transform2D* компоненти током сваког кадра.
- Праћење перформанси: Апликација континуирано рачуна и приказује *ФПС* у наслову прозора.

Евалуациона апликација показује како се систем рендеровања носи са великим бројем спрајтова, и колико ефикасно *ЕКС* језгро процесира ажурирања позиција пре него што се подаци проследе графичком процесору.

6.2 Техничка имплементација евалуације

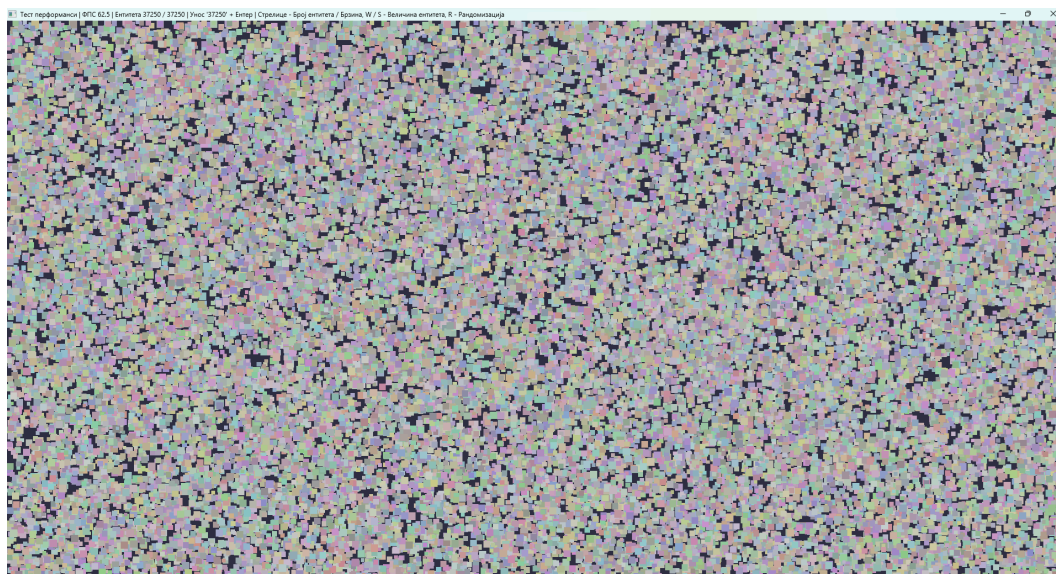
Језгро евалуационе апликације чини структура *PerfScene*, која управља стањем симулације и *ЕКС* светом (*World*). За сваки ентитет у евалуацији, поред основних компоненти (*Transform2D*, *SpriteComponent*), придружена је и компонента *Velocity2D* која дефинише вектор брзине. Логика симулације је смештена у методу *simulate*, која у сваком кадру ажурира позиције ентитета на основу њихове брзине, времена које је протекло од претходног кадра (*dt*), и глобалног фактора убрзања. Приликом ажурирања, примењује се и једноставна детекција колизија са ивицама прозора. Када ентитет дотакне границу екрана, вредност његове компоненте брзине се обрне, што обезбеђује да сви спрајтови остану унутар видљивог подручја.

Прецизно мерење перформанси реализује се коришћењем “клизећег прозора” времена. Унутар структуре *PerfScene* прате се два бројача: *fps_timer* (акумулирано време у секундама) и *fps_frames* (бројач кадрова). Сваких 0,25 секунди, апликација израчунава вредност *ФПС*-а према формули $\text{fps_frames} / \text{fps_timer}$. Након овог интервала, бројачи се ресетују, чиме се обезбеђује стабилан и читљив приказ перформанси, без наглих скокова који би били присутни код мерења појединачних кадрова. Ова вредност се затим приказује у наслову прозора, који се ажурира тек када је то неопходно, како би се смањио трошак системских позива.

6.3 Анализа перформанси

Евалуациона апликација је покренута у *release* моду користећи `cargo run -r`, на *Windows 11* систему са *RTX 3050 6GB Laptop GPU* и *Intel Core i5-13420H* процесором.

Евалуациона апликација приказује да *Rusty* погон за видео игре у ~60 *ФПС*-а подноси 37000 ентитета различите величине који се крећу насумично по прозору.



Слика 1: Приказ евалуационе апликације са 37000 ентитета

На екстремној евалуацији са 500 хиљада ентитета, апликација тече у ~2 ФПС-а, иако се користи ~25% капацитета графичке картице уређаја на ком је покренута, као и свега 10% процесора. То имплицира да је *Rusty* погон за видео игре могуће унапредити паралелном вишенимном обрадом података и оптимизованијим рендеринг системом. Свакако, више десетина хиљада ентитета у дводимензионалном простору који теку у ~60 ФПС-а су задовољавајуће перформансе за једнонитни систем који прати основне ЕКС принципе и користи основе *wgri*-а.

Ова мерења указују на то да је *Rusty* стабилна библиотека која се може користити за израду мање комплексних видео игара и стабилних симулација са до пар десетина хиљада ентитета у једном тренутку.

У овом раду представљена је имплементација погона за видео игре користећи *ЕКС* архитектуру у програмском језику *Rust*, заједно са интеграцијом *wgpu* библиотеке за рендеровање. Кроз анализу и практичну реализацију показано је да је могуће изградити ефикасан, модуларан и, пре свега, безбедан систем који користи предности *Rust*-овог модела власништва и *wgpu* апстракције над графичким АПИ-јевима (*Vulkan*, *Metal*, *DX12*).

Хипотеза да је могуће направити безбедан *ЕКС* и рендеринг слој користећи *Rust* уз *wgpu* је успешно потврђена. *Rust*-ов систем типова и провера животног века омогућили су да се избегну уобичајене грешке у раду са меморијом, док је *wgpu* обезбедио платформу за преносиво графичко програмирање.

7.1 Могућности за проширење система

Иако је тренутна имплементација функционална, постоји простор за даља унапређења и проширења:

- *ЕКС* систем: Тренутни *ЕКС* подржава основне операције над ентитетима и компонентама. Систем би се могао проширити подршком за паралелно извршавање система што би додатно побољшало перформансе. Такође, имплементација напреднијих структура за складиштење компоненти (нпр. *Sparse Sets* за брзе упите) би додатно оптимизовала приступ подацима.
- Рендеринг опције: Рендеринг слој се може проширити имплементацијом напреднијих концепата попут динамичког осветљења, сенки, и пост-процесирања (нпр. *bloom*, *depth of field*). Подршка за слој корисничког интерфејса би такође била драгоцену за развој видео игара и других алата.
- Интеграција скрипти: Додавање подршке за скриптне језике попут *Rhai* или *Lua* би омогућило дефинисање логике игре без потребе за поновним компилирањем изворног кода.

7.2 Конкретна примена у виду развоја видео игре

Систем је дизајниран као модуларна основа, што омогућава да се на њему лако изгради конкретна видео игра. Процес развоја игре би подразумевао:

1. Управљање ресурсима: Имплементацију система за асинхронно учитавање текстура, модела и звукова.
2. Физика: Интеграцију постојеће *ЕКС* архитектуре са *physics* погоном (нпр. *Rapier*) ради детекције колизија и симулације кретања.
3. Управљање сценом: Креирање хијерархијских структура над *ЕКС*-ом ради лакшег управљања објектима у сцени.
4. Унос: Повезивање *winit* библиотеке са логиком игре за обраду уноса са тастатуре, миша и џојстика.

Овај рад потврђује да *Rust* и *wgpu* представљају модеран, сигуран и моћан избор за развој погона за игре, пружајући добру основу за даљи рад на сложенијим софтверским системима.

Списак слика

Слика 1 Приказ евалуационе апликације са 37000 ентитета	28
---	----

Списак листинга

Списак коришћених скраћеница

Скраћеница	Опис
EKC / ECS	Entity Component System (ентитет-компонента-систем архитектура)
АПИ / API	Application Programming Interface (апликациони програмски интерфејс)
ФПС / FPS	Frames Per Second (број кадрова у секунди)
JSON	JavaScript Object Notation (формат за размену података)
UUID	Universally Unique Identifier (универзално јединствени идентификатор)
FIFO	First In, First Out (први улази, први излази)
SIMD	Single Instruction, Multiple Data (једна инструкција, више података)
SoA	Structure of Arrays (структура низова)
RGBA	Red, Green, Blue, Alpha (црвена, зелена, плава, алфа)
FFI	Foreign Function Interface (интерфејс за комуникацију са страним програмским језицима)
WGSL	WebGPU Shading Language (језик за писање шејдера за WebGPU)
UI	User Interface (кориснички интерфејс)

Списак коришћених појмова

Појам	Објашњење
Погон за видео игре	Софтверски систем који обезбеђује инфраструктуру за развој игара (рендеринг, физика, улази).
ЕКС / ECS	Архитектурални образац за развој игара заснован на ентитетима, компонентама и системима.
Ентитет	Логички објект у ЕКС моделу који представља јединствени идентификатор без уграђеног понашања.
Компонента	Структура података која описује један аспект стања ентитета (нпр. позиција, здравље).
Систем	Логика која обрађује скупове ентитета који поседују одређене компоненте.
Небезбедан код	Део Rust кода чију исправност компајлер не може статички да верификује.
Data-oriented дизајн	Приступ пројектовању софтвера који организацију података ставља у први план.
Власништво	Rust механизам управљања меморијом који осигурава да свака вредност има једног власника.
Позајмљивање	Rust механизам приступа вредностима без преноса власништва (референце).
Животни век	Rust концепт који прати колико дуго референце на податке остају валидне.
Серијализација	Процес претварања структура података у формат погодан за чување или пренос (нпр. JSON).
Trait	Rust механизам за дефинисање заједничког понашања различитих типова.
Marker trait	Trait без метода, који служи за означавање својстава типа (нпр. да је тип компонента).
Type erasure	Техника скривања конкретних типова иза заједничког интерфејса ради хетерогеног складиштења.
Dynamic dispatch	Механизам позивања функција током извршавања програма коришћењем полиморфних показивача.
Sparse set	Структура података за ефикасно управљање компонентама уз брзу итерацију и проверу.
SoA	Structure of Arrays, организација података где се свако поље чува у засебној колони или низу.
FIFO	First In, First Out, принцип реда код ког се први убачени елемент први обрађује.
JSON	Формат за размену података заснован на тексту.
WGPU	Rust имплементација WebGPU стандарда за хардверски убрзан рендеринг.
Bind groups	WGPU ресурси који омогућавају шејдерима приступ подацима (текстуре, униформе).
Render pass	Фаза рендеровања у WGPU-у у којој се дефинишу циљеви цртања и команда цртања.
Pipeline	WGPU конфигурација која дефинише стање графичког процесора (шејдери, структура података).
Buffer	Меморијски простор на графичком процесору за чување података (темена, индекси).
WGSL	WebGPU Shading Language, језик за писање шејдера на графичком процесору.

Биографија

Никола Огњеновић је рођен 03.10.2003. године у Шапцу где је завршио основну школу и гимназију са одличним успехом.

Факултет техничких наука у Новом Саду, смер Софтверксо инжењерство и информационе технологије (ОАС), уписао је 2022. године. Положио је све испите са просечном оценом 9,39/10.

Током студија завршио је 3 праксе, радио је у 2 домаће фирме као *full stack* веб програмер и држао вежбе на факултету у Виљнусу.

Литература

- [1] R. Lord, „Evolve your hierarchy“. Приступљено: 30. Мај 2026. [На Интернету]. Доступно на <https://www.richardlord.net/blog/ecs/what-is-an-entity-framework.html>
- [2] Unity Technologies, „Unity's Data-Oriented Technology Stack (DOTS)“. Приступљено: 29. Мај 2026. [На Интернету]. Доступно на <https://unity.com/dots>
- [3] Epic Games, „Gameplay Framework in Unreal Engine“. Приступљено: 29. Мај 2026. [На Интернету]. Доступно на <https://dev.epicgames.com/documentation/unreal-engine/gameplay-framework-in-unreal-engine?lang=en-US>
- [4] Bevy Foundation and contributors, „bevyengine/bevy: A refreshingly simple data-driven game engine built in Rust“. Приступљено: 29. Мај 2026. [На Интернету]. Доступно на <https://github.com/bevyengine/bevy>
- [5] Bevy contributors, „Crate bevy - docs.rs“. Приступљено: 29. Мај 2026. [На Интернету]. Доступно на <https://docs.rs/bevy>
- [6] S. Klabnik и C. Nichols, „The Rust Programming Language: Unsafe Rust“. Приступљено: 29. Мај 2026. [На Интернету]. Доступно на <https://doc.rust-lang.org/book/ch20-01-unsafe-rust.html>
- [7] R. Fabian, *Data-Oriented Design*. The Pragmatic Programmers, 2020.
- [8] Bevy contributors, „Bevy entity documentation“. Приступљено: 01. Јуни 2026. [На Интернету]. Доступно на <https://docs.rs/bevy/latest/bevy/prelude/struct.Entity.html>
- [9] M. Klein, M. Suijten, и contributors, „ash: A very lightweight wrapper around Vulkan“. Приступљено: 03. Јуни 2026. [На Интернету]. Доступно на <https://github.com/ash-rs/ash>
- [10] Vulkan Contributors, „vulkano: Safe and rich Rust wrapper around the Vulkan API“. Приступљено: 03. Јуни 2026. [На Интернету]. Доступно на <https://vulkano.rs/>
- [11] J. Oster и contributors, „pixels: A tiny hardware-accelerated pixel frame buffer“. Приступљено: 03. Јуни 2026. [На Интернету]. Доступно на <https://github.com/parasyte/pixels>
- [12] rust-windowing Contributors, „softbuffer: Cross-platform software buffer for drawing pixels“. Приступљено: 03. Јуни 2026. [На Интернету]. Доступно на <https://github.com/rust-windowing/softbuffer>
- [13] T. ggez team, „ggez: A lightweight game framework for making 2D games with minimum friction“. Приступљено: 03. Јуни 2026. [На Интернету]. Доступно на <https://ggez.rs/>
- [14] F. Logachev и contributors, „macroquad: Simple and easy to use game library for Rust“. Приступљено: 03. Јуни 2026. [На Интернету]. Доступно на <https://macroquad.rs/>
- [15] raylib-rs contributors, „raylib-rs: Rust bindings for raylib“. Приступљено: 03. Јуни 2026. [На Интернету]. Доступно на <https://github.com/deltaphc/raylib-rs>
- [16] wgpu contributors, „wgpu: A cross-platform, safe, pure-Rust graphics API“. Приступљено: 03. Јуни 2026. [На Интернету]. Доступно на <https://wgpu.rs/>